

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла):

студент групи AI-245
Дуднік Захар Романович

Перевірив:

ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	577	5
Час, с	10,067	4,048
Токенів/с	96,41	151,685

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1441	165
Час, с	16,2	1,97

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Усі 577 токенів пішли на прихований внутрішній монолог (Chain-of-Thought) всередині службових тегів `<think>`, де модель аналізувала обмеження запиту та перевіряла факт. Інтерфейс вивів лише фінальне слово «Київ», проте апаратний лічильник зафіксував повний обсяг згенерованого тексту міркувань.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі незамінні у складних багатокрокових завданнях — математиці, формальній логіці, а також аналізі та дебагу коду, де критично

необхідна самоперевірка. Натомість їхнє використання невиправдане для прямого пошуку фактів, базової класифікації тексту чи швидких чат-ботів, де приховані роздуми лише створюють зайву затримку та марнують ресурси.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі навчають методами SFT та RLHF на зразках готових ідеальних відповідей, тренуючи їх одразу надавати лаконічний кінцевий результат. Міркувальні моделі навчають через масштабне навчання з підкріпленням (RL) із перевірюваними винагородами, заохочуючи систему формувати довгі ланцюжки роздумів, тестувати гіпотези та виправляти власні помилки на льоту.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість токенів вказує лише на використання додаткових обчислень під час інференсу (test-time compute), а не на більший обсяг фундаментальних знань. Для простих питань це звичайне «перемудровування» (overthinking), хоча в орієнтованих на логіку задачах такий алгоритмічний пошук дійсно дозволяє отримувати значно точніші рішення.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Llama3.2:3b закономірно випереджає qwen3:4b. Головна причина полягає в обмеженні пропускну здатності пам'яті (memory bandwidth): інференс вимагає зчитувати всі ваги моделі для кожного окремого токена, тому легша 3B-модель виконує менше пересилань даних на крок, ніж 4B. Додатково на користь Llama грають архітектурні фактори — ефективна оптимізація механізму Grouped-Query Attention (GQA) та менший розмір шарів у порівнянні з роздутішою архітектурою Qwen.

Висновок за завданням 1

Експеримент показав, що два показники — швидкість генерації та загальний час відповіді — не можна плутати. Перший характеризує обладнання й розмір моделі, другий — обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати

відрізняються у 115 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 129,6 с проти 15,8 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3 – Вплив system повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	Лаконічний асистент	1343	суржик з англійською та галюцинаціями	12	Ні
llama3.2:3b	українською ,2 речення	289	арабські літери	2	Частково
qwen3:4b	Лаконічний асистент	357	Українська	5	Так
qwen3:4b	українською ,2 речення	151	Українська	1	замість двох речень згенеровано одне складне

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	Лаконічний асистент	59	403	462
llama3.2:3b	українською, 2 речення	63	79	142
qwen3:4b	Лаконічний асистент	65	2466	2531
qwen3:4b	українською, 2 речення	74	944	1018

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві моделі вклалися в ліміт, проте qwen3:4b впоралася значно точніше й осмисленіше, сформулювавши повноцінні граматичні речення без втрати змісту. Модель llama3.2:3b формально теж видала 1–2 речення, але суттєво спотворила їхній синтаксис та логіку.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку? Так, модель llama3.2:3b систематично порушувала, вставляючи англійські терміни, русизми та навіть арабські літери. Модель qwen3:4b щоразу відповідала коректною українською.

3. Як змінився prompt_tokens при зміні system повідомлення і чому?

Показник prompt_tokens зріс для обох моделей, оскільки інструкція «українською, 2 речення» довша за кількістю символів, ніж «лаконічний асистент». Додатковий приріст зумовлений тим, що кириличний текст розбивається токенизатором на більшу кількість субтокенів у порівнянні з латиницею.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

qwen3:4b є міркувальною моделлю, яка перед формуванням відповіді будує розлогий ланцюжок прихованих роздумів. `completion_tokens` фіксує всі згенеровані токени включно з цим внутрішнім монологом, тоді як звичайна інструктивна модель Llama генерує лише видимий текст.

Відповіді на питання (крок 7)

1. Чому моделі по різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Різниця зумовлена якістю навчального корпусу та оптимізацією токенизатора: серія Qwen має набагато краще представлення кирилиці та багатомовних даних, ніж англоцентрична LLaMA. Крім того, міркувальні моделі перед генерацією планують структуру тексту у внутрішньому ланцюжку думок, що допомагає чітко тримати рамки обмежень, тоді як компактні інструктивні моделі без попереднього етапу міркування легше «гублять» обмеження довжини.

2. Чим `system` повідомлення відрізняється від `user` повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: Для архітектури трансформера обидва типи повідомлень — це єдиний потік токенів, просто розділений спеціальними службовими тегамі ролей під час шаблонізації пром프트. Під час налаштування (RLHF/DPO) моделі навчають сприймати системні інструкції як фундаментальні правила з вищим пріоритетом над репліками користувача. Проте цей пріоритет не є абсолютним: через природний механізм уваги останні вхідні дані (`user`) мають високу вагу «свіжості» і часто здатні частково перекрити системний тон.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Міркувальні моделі витрачають токени на прихований внутрішній монолог (Chain-of-Thought, зазвичай всередині службових тегів на кшталт `<think>`), де детально прораховують відповідь перед її публікацією. Ці проміжні міркування повністю тарифікуються та зараховуються до вихідних токенів (`completion_tokens`), навіть якщо вони приховані інтерфейсом. Саме тому у Qwen зафіксовано сотні й тисячі токенів виводу, хоча фінальний видимий текст складався лише з 1–5 речень

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: У компактних моделях найчастіше спрацьовує ефект свіжості (recency bias), тому пріоритет зазвичай отримує остання інструкція від user. У разі прямого жорсткого конфлікту мала модель нерідко втрачає узгодженість: вона може змішати команди, почати повторюватися або перейти в галюцинації. Великі та добре налаштовані моделі дотримуються system лише тоді, коли запит користувача порушує глобальні обмеження або правила безпеки.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Через невелику кількість параметрів малі моделі мають обмежену місткість пам'яті й оптимізовані переважно під домінуючу англійську мову, маючи значно слабший зв'язок між поняттями в українській кирилиці. Коли модель змушена одночасно контролювати зміст, ліміт речень та нерідну мову, виникає перевантаження контекстних ваг. Як наслідок, трапляється «декодерний збій», коли модель зісковзує в суржик, англійські вставки або навіть помилково вибирає сусідні токени з зовсім інших писемностей (наприклад, арабські символи у LLaMA 3.2).

Висновок за завданням 2

Експеримент довів, що системне повідомлення є лише м'яким вектором поведінки моделі, а не надійним програмним обмеженням. Для компактних моделей самої текстової інструкції категорично недостатньо: особливості кириличної токенизації призводять до регулярного зриву лімітів на обсяг речень чи збереження потрібної мови. Системний промпт задає тональність, але виявляється безсилим проти надлишкового міркування чи мовних артефактів без жорсткої фіксації контексту. Надійність керування визначається не довжиною промпту в system, а суворими технічними обмеженнями.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	3.48	15.45
completion_tokens	90	147
Довжина відповіді, символів	259	447
Мова відповіді	суржик, змішування з англійською	літературна українська мова
Фактичних помилок у відповіді	назва HDD «масштабованою пам'яттю», граматичний брак «Оперативна пам'яті»	0

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Завдяки єдиному інтерфейсу OpenAI API знадобилося змінити лише 1 рядок коду.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці?

Відповідь хмарної моделі gemini-3.5-flash-lite значно точніша та структурніша. Вона використала вдалі аналогії («робочий стіл» проти «шафи для документів») і чітко пояснила ключовий фактор енергонезалежності: «Вимкнув комп'ютер — усе на "робочому столі" зникло». Локальна ж модель звела все до примітивної тавтології про дані, які «Currently використовуються» та «не Currently використовуються».

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель продемонструвала значні проблеми в обох аспектах. Русизм («временная память»), граматична помилка у відмінку в першому ж слові («Оперативна пам'яті») та змішування регістрів з англійськими вкрапленнями. Модель невдало охарактеризувала HDD як «масштабовану пам'ять» і не змогла коректно сформулювати поняття енергозалежності

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хоча в конкретному вимірі хмара зазнала затримки (15.45 с проти 3.48 с), у типових умовах хмарні моделі генерують відповідь швидше за рахунок серверного заліза.

Відповіді на питання (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI здобула перевагу першопрохідця, тому найпопулярніші бібліотеки та фреймворки (як-от LangChain чи LlamaIndex) першочергово проєктувалися під ендпоінт `/v1/chat/completions`. Підтримка цього інтерфейсу іншими провайдерами та сервером Ollama дає змогу розробникам змінювати бекенд між локальними й хмарними моделями простим оновленням `base_url` без переписування кодової бази.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Кількість параметрів критично визначає місткість «знань про світ», здатність до логічного міркування та якість роботи з неанглійськими мовами, через що 3B моделі часто переходять на суржик і плутають терміни. Хоча архітектура, якість тренувального датасету та пост-тренінг (RLHF) суттєво впливають на результат, малі моделі фізично не мають достатнього об'єму пам'яті для глибокого розуміння складного контексту на рівні 70B аналогів.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарні провайдери розгортають моделі на промислових прискорювачах (наприклад, Nvidia H100 або Google TPU) із пропускнуою здатністю пам'яті в терабайти на секунду та використовують оптимізовані рушії інференсу (спекулятивне декодування, vLLM). Споживчі комп'ютери обмежені значно повільнішою RAM або VRAM, через що хмарні сервери генерують токени настільки швидко, що це з запасом нівелює затримку мережевого пінгу

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Головні загрози — це витік персональних даних (PII), вихід комерційної таємниці за межі компанії та потенційне використання вхідних даних для донавчання майбутніх моделей провайдера. Для захисту комерційні проєкти укладають Enterprise-угоди з політикою нульового збереження даних (Zero Data Retention), впроваджують локальну анонімізацію/маскування тексту перед надсиланням або орендують моделі в ізольованій приватній хмарі (VPC).

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це встановлене провайдером обмеження кількості запитів (RPM) або токенів (TPM) за хвилину чи добу для запобігання перевантаженню безкоштовної інфраструктури. Розробники вирішують цю проблему через впровадження черг із повторними спробами (exponential backoff), агресивне кешування частих відповідей, а також налаштування динамічного перемикавання (fallback) між пулом із кількох API-ключів або альтернативними моделями.

Висновок за завданням 3

Експеримент довів, що хоча використання уніфікованого інтерфейсу дозволяє миттєво перемикатися між локальними та хмарними моделями, обсяг параметрів залишається критичним фактором якості. llama3.2:3b виявилася нездатною надійно працювати з українською мовою, продукуючи суржик, граматичні та фактичні

хиби. Натомість хмарна gemini-3.5-flash-lite продемонструвала здатність до глибокого міркування, надавши розгорнуту та точну відповідь літературною мовою без жодної помилки. Попри те, що локальний сервер спрацював швидше в ізольованому тесті, потужне серверне обладнання хмарних провайдерів з надлишком компенсує мережеві затримки на масштабних задачах. Відтак, надійність та якість результату визначаються фізичною місткістю моделі, роблячи хмарні рішення безальтернативними для складних завдань, тоді як малі локальні моделі залишаються компромісом винятково для приватності.

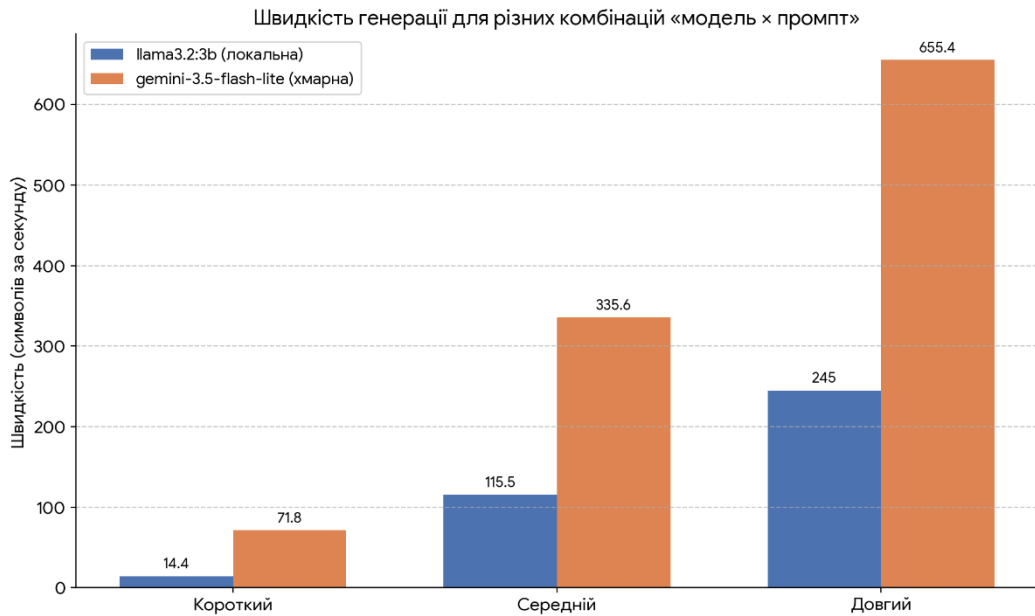
Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 6 – Порівняння локальної та хмарної моделі

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.058	2.168	14.4
локальна	llama3.2:3b	середній	2.061	2.9	115.5
локальна	llama3.2:3b	довгий	2.086	7.07	245
хмарна	gemini-3.5-flash-lite	короткий	0.708	0.919	71.8
хмарна	gemini-3.5-flash-lite	середній	0.731	1.426	335.6
хмарна	gemini-3.5-flash-lite	довгий	0.754	4.918	655.4

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.053	2.058
Всього, с	2.494	2.168



Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Найкращий показник TTFT має хмарна модель gemini-3.5-flash-lite. Це пов'язано з тим, що вона працює на серверних кластерах, здатних ініціалізувати й обробляти дані значно швидше за звичайне домашнє обладнання.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть?

Так, збігається, оскільки хмарна модель gemini-3.5-flash-lite лідирує як за швидкістю старту, так і за кількістю згенерованих символів за секунду на всіх промптах.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

При збільшенні обсягу промпту швидкість генерації зростає в обох моделях, але розрив між ними скорочується: від переваги хмари у 5 разів на короткому до 2.5 раза на довгому запиті.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Час виконання може суттєво коливатися через фонові процеси локальної системи, навантаження на хмарні сервери або нестабільність інтернет-з'єднання. Саме тому одного запуску категорично недостатньо для об'єктивної оцінки.

5. Чому холодний старт повільніший за теплий?

Теоретично, холодний старт має бути повільнішим, оскільки системі потрібен час, щоб фізично зчитати ваги моделі з постійного накопичувача в оперативну або відеопам'ять, тоді як при теплому старті вони вже там знаходяться. Проте в результатах TTFT залишався незмінно стабільним (~2.05–2.09 с). Це пояснюється тим, що модель Llama3.2:3b має невеликий розмір (2 ГБ), і сучасний швидкий SSD-накопичувач зчитує її за доли секунди. Крім того, Windows дуже агресивно кешує файли: навіть запуск 8B-моделі (qwen3:8b) міг не повністю витіснити файли Llama з резерву оперативної пам'яті, якщо в системі її достатньо. Зрештою, ці стабільні ~2 секунди витрачаються не на читання з диска, а на системну ініціалізацію движка Ollama, виділення пам'яті та математичне обчислення першого токена, що займає однаковий час як для холодного, так і для теплового старту.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT (Time To First Token) система виконує токенизацію запиту та проходить фазу prefill. У цій фазі модель паралельно обчислює весь вхідний текст для створення початкового KV-кешу та генерації найпершого токена. Для хмарних моделей сюди також входить час на встановлення мережевого з'єднання та обробку черги на сервері. Цей показник вимірюють окремо, оскільки він відображає стартову чутливість системи (latency) — час очікування користувача до появи першої реакції. Фаза генерації (написання тексту) є іншим обчислювальним процесом.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Гірший TTFT у хмарної моделі зазвичай зумовлений мережевими затримками (ping, маршрутизація, черги доступу до API), яких немає при локальному запуску. Проте щойно починається генерація (фаза decode), хмарні сервери використовують масиви промислових прискорювачів (Enterprise

GPU/TPU) з величезною пропускнуою здатністю пам'яті. Вони здатні видавати токени набагато швидше, ніж споживче локальне "залізо". (Варто зазначити, що у вашій Таблиці 6 хмарна модель має і кращий TTFT, і кращу швидкість).

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт — це ситуація, коли гігабайти ваг моделі знаходяться на повільному постійному накопичувачі (SSD або HDD). Під час першого запуску операційна система повинна перенести ці дані в оперативну пам'ять (RAM) або відеопам'ять (VRAM), що займає багато часу. При наступних запусках (теплий старт) ваги вже завантажені у швидку пам'ять, тому час витрачається лише на інференс. У Таблиці 7 файлу це відображено у зменшенні загального часу виконання (з 2.494 с до 2.168 с).

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіана стійка до аномальних викидів (outliers). На швидкість окремого запуску можуть вплинути непередбачувані фактори: системні переривання, фонові процеси ОС, тротлінг через нагрів або раптові стрибки пінгу (для хмарних моделей). Середнє арифметичне значення може сильно спотворитися через один випадковий дуже довгий запит, тоді як медіана показує найбільш стабільний і типовий для системи результат.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Обробка вхідного запиту (prefill) відбувається паралельно. Завдяки архітектурі Transformer, модель множить матриці для всіх токенів промπτу одночасно, ефективно завантажуючи всі обчислювальні блоки відеокарти чи процесора. Натомість генерація (decode) є авторегресивним процесом — модель має згенерувати перший токен, додати його до контексту, потім згенерувати другий і так далі. Цей процес послідовний, його неможливо розпаралелити в часі, і він сильно обмежений швидкістю доступу до пам'яті.

Висновок за завданням 4

Експеримент довів, що обчислювальні потужності базової інфраструктури є критичним фактором швидкодії мовних моделей. Локальна llama3.2:3b продемонструвала стабільну, але суттєво нижчу швидкість генерації та вищий час до появи першого токена (TTFT), який виявився обмеженим системними накладними витратами рушія, а не дисковим читанням завдяки агресивному кешуванню на рівні ОС. Натомість хмарна gemini-3.5-flash-lite продемонструвала беззаперечну перевагу, миттєво ініціалізуючись та генеруючи текст у 3–5 разів швидше на всіх типах промптів. Попри те, що при збільшенні обсягу запиту локальна модель здатна розігнатися і відносний розрив дещо скорочується, потужне серверне обладнання хмарних провайдерів з надлишком компенсує будь-які мережеві затримки. Відтак, продуктивність системи безпосередньо залежить від масштабу апаратних ресурсів, роблячи хмарні рішення безальтернативними для завдань, що вимагають високої пропускної здатності та миттєвого відгуку, тоді як малі локальні моделі залишаються апаратним компромісом винятково для офлайн-роботи та забезпечення абсолютної приватності.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 8 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	2	2	2
2. Формат	2	2	2
3. Стиснення	0	1	2
4. Код	1	2	1
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	5	10	11

Таблиця 9 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	так	так	ні
4. Код	так	ні	так
5. Логіка	так	так	ні
6. Українська	так	ні	ні

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала Хмарна модель, а основна різниця порівняно з локальними моделями виникла у завданнях на стиснення тексту, логіку та якість української мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Оцінки двох прогонів розійшлися здебільшого у локальних моделей на завданнях 3–6, тоді як Хмарна модель мала лише розбіжність у генерації коду. Це свідчить про низьку надійність невеликих моделей, чиї результати, формат та логіка можуть випадковим чином ламатися від прогону до прогону.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Усі три моделі успішно виконали цю вимогу, згенерувавши чистий JSON.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

У локальних моделей були серйозні проблеми з якістю тексту. Модель llama3.2:3b використовувала русизми та вигадані слова, а також обидві локальні моделі в окремих прогонах повністю переходили на російську мову під час міркувань або стиснення.

Приклад провалу (крок 7)

1. Повний текст запиту:

"8 команд, кожна з кожною по разу. Скільки матчів? При 4 матчах на день — скільки днів? Покажи хід розв'язання."

2. Повний текст відповіді:

1)"Давайте разберемся в цьому проблемі шаг за шагом.\n\n1. Молимося відомо, скільки команд та кожна команда грає по разу.\n2. Важно зазначити, що в задачі не задавана кількість матчів, які грають команди по разу.\n3. Затим, як ми знаємо, що команд у 8, та кожна команда грає по разу, то для розповсюдження команд по 8 по 1 команду по разу, потрібно 8 матчів.\n4. Нам теж повідомляється, що команди грають по 4 матчі на день, а це означає, що по 4 матчі грають на день.\n5. Оскільки команди грають по 4 матчі на день, то за 1 день команди грають 4 матчі.\n6. Для розповсюдження команд по 8 по 1 команду по разу, потрібно 8 матчів.\n7. Оскільки команди грають по 4 матчі на день, то за 1 день команди грають 4 матчі.\n8. Для розповсюдження команд по 8 по 1 команду по разу, потрібно 8 матчів/день * 8 днів = 64 днів.\n9. Таким образом, для розповсюдження команд по 8 по 1 команду по разу, потрібно 64 днів.\n\n0Конечательный ответ - 64 дня."

[illegible]

[illegible]

3. Яка правильна відповідь мала бути:

Загалом у турнірі відбудеться 28 матчів ($8 * 7 / 2$). Оскільки щодня проходить по 4 матчі, для завершення турніру знадобиться 7 днів ($28 / 4$).

4. Пояснення, чому так сталося:

Коли модель не змогла згадати формулу комбінаторики і спробувала вирішити задачу описово, вона втратила контекст розрахунку. Оскільки мовні моделі генерують текст токен за токеном на основі ймовірностей попереднього контексту, увага моделі зациклилася на помилковій думці, яку вона щойно згенерувала. Ймовірність видати ту саму фразу знову стала математично найвищою, що призвело до нескінченного повторення.

Відповіді на питання (крок 8)

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Окрім оцінювання вручну за кастомною рубрикою, в індустрії та дослідженнях застосовують:

Стандартизовані бенчмарки (Automated Benchmarks): Набори фіксованих тестів для вимірювання конкретних навичок: MMLU / MMLU-Pro (загальні знання та міркування), GSM8K / MATH (математика), HumanEval (генерація коду), ARC (логіка).

LLM-as-a-Judge (Оцінювання сильнішою мовною моделлю): Використання передових моделей (наприклад, GPT-4 або Claude 3.5 Sonnet) для автоматичного суддівства відповідей менших моделей за шкалою чи в режимі парного порівняння (pairwise comparison).

Краудсорсингові арени (Human Preference / Elo rating): Порівняння моделей «наосліп» реальними користувачами, де кожна відповідь отримує голос, формуючи рейтинг за системою Ело (найвідоміший приклад — LMSYS Chatbot Arena).

Метрики для RAG-систем: Спеціалізовані метрики (наприклад, фреймворки Ragas або TruLens) для перевірки відповідності контексту

(Faithfulness), релевантності відповіді (Answer Relevance) та точності вилучення (Context Precision).

Традиційні NLP-метрики схожості: ROUGE, BLEU, BERTScore — для задач перекладу та самаризації, де є чіткий еталонний текст.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Сильний патерн у навчальних даних: Більшість пар «запит на код/JSON → відповідь» у датасетах fine-tuning (SFT) та діалогових корпусах відформатовані саме через кодові блоки Markdown (json ...). Цей статистичний зв'язок є настільки домінантним, що модель слідує йому за інерцією.

Специфіка System Prompt платформ: Багато чат-інтерфейсів містять у прихованому системному промпті загальну настанову на кшталт: «завжди оформлюй код і структуровані дані у Markdown-блоки», що вступає в прямий конфлікт із призначенням для користувача запитом.

Слабка увага до заперечень: Конструкції з часткою «не» (від'ємні обмеження на кшталт «НЕ використовуй markdown») обробляються моделями гірше, ніж позитивні інструкції («поверни лише чистий сирий текст»).

Вирішення на практиці: Застосування спеціальних режимів API (JSON mode або Structured Outputs / JSON Schema) або граматик (GBNF) при локальному інференсі, які апаратно маскують недозволені токени на рівні вибірки логітів.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Дисбаланс навчального корпусу: Масив україномовних текстів у корпусах попереднього навчання (pre-training) зазвичай становить менше ніж 0.1–1% від загального обсягу, тоді як англійська займає переважну більшість.

Неефективна токенизація: Токенізатори здебільшого оптимізовані під латиницю. Українські слова часто розбиваються не на цілі слова чи морфеми, а на окремі байти чи фрагменти літер (по 2–4 токени на коротке слово). Через це контекстне вікно витрачається швидше, а моделі складніше формувати смислові та граматичні зв'язки.

Обмежена ємність параметрів (Parameter Capacity): У моделях розміром 3B–4B параметрів вистачає переважно на збереження знань і синтаксису домінантних мов. Внутрішній процес міркування часто відбувається внутрішнім представленням англійської мови, що призводить до калькування, русизмів або порушення відмінювання при декодуванні в українську.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Стохастичне семплювання (Sampling): Моделі генерують не жорстко детермінований текст, а ймовірнісний розподіл для наступного токена. Якщо параметр температури ($Temperature > 0$) або параметри top-p / top-k не зафіксовані на нульових значеннях, вибір наступного слова здійснюється випадковим чином з пулу найбільш імовірних варіантів. Ефект лавини (Compounding Errors): Навіть якщо на першому кроці модель обрала синонім замість первинного слова, цей новий токен змінює весь наступний контекст, через що решта відповіді може розвиватися за зовсім іншою траєкторією якості та логіки. Асинхронність обчислень на GPU: Навіть при $Temperature = 0$ деякі оптимізації багатопотокових обчислень (наприклад, порядок зведення операцій у рушіях інференсу) можуть спричиняти мінімальні математичні похибки у десяткових дробах логітів, що іноді призводить до зміни вибору пограничного токена.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Відповідь: Галюцинація — це генерація моделлю тверджень, які граматично правильні, стилістично впевнені та звучать правдоподібно, але є фактологічно неправдивими або суперечать наданому контексту.

Модель радше вигадас факт через такі особливості архітектури й навчання:

Відсутність концепції «знання»: Модель є авторегресійним апроксиматором імовірності наступного токена, а не базою даних із перевіркою істинності. Її мета — продовжити текст найімовірнішим шляхом, а не перевірити достовірність.

Штраф за відмову під час RLHF/SFT: На етапі fine-tuning моделі часто заохочували бути «корисними помічниками» (Helpfulness), караючи за відмови («я не знаю»). У результаті модель прагне заповнити запит правдоподібними сутностями замість зупинки генерації.

Неможливість відкалібрувати власну впевненість: Внутрішні стани моделі слабо корелюють із реальним доступом до факту. Якщо у внутрішній вазі немає точної відповіді, механізм уваги просто витягує семантично найближчі асоціації.

Висновок за завданням 5

Експеримент довів, що масштаб моделі критично визначає її надійність. Через нестачу даних локальна llama3.2:3b продемонструвала нестабільність та схильність до зациклень. Натомість хмарна gemini-3.5-flash-lite показала беззаперечну перевагу, генеруючи безпомилкові та стилістично вивірені відповіді. Хоча qwen3:4b здатна конкурувати у базовому кодингу, хмарні рішення є безальтернативними для складної логіки, залишаючи малим локальним моделям лише нішу простого парсингу та забезпечення приватності.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	5	—
temperature	1.5	5	0
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0
Унікальних відповідей	2
Усі відповіді однакові (так / ні)	ні

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для генерації JSON слід встановити `temperature = 0.0`, оскільки це забезпечує максимальну детермінованість.

2. Яке — генератору варіантів заголовків?

Для генератора заголовків оптимальним є значення `temperature = 0.7–0.8`, що надає моделі творчу свободу для різноманітності ідей без ризику втратити логіку.

3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою?

`temperature` плавно змінює весь профіль розподілу ймовірностей, тоді як `top_p` жорстко й динамічно відсікає хвіст малої ймовірності за порогом сумарної ймовірності, адаптуючи вибірку до ступеня впевненості моделі в контексті.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

`temperature = 0` не гарантує абсолютної відтворюваності через недетермінізм операцій із рухомою комою. Експеримент показав розбіжність, згенерувавши 2 унікальні варіанти на 10 запусків через різницю в одному символі — 9 разів дефіс (-) та один раз тире (—).

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона взагалі могла з'явитися.

Найбезглуздішою стала відповідь із суцільним псевдомовним маренням: «Бігай, скерую собі шайки на електродах у лазерних масках, а золоїмаси на журових

гусенахідках в сітках бідного кумафа...». За значення 1.5 розподіл ймовірностей майже повністю зрівнюється, через що модель починає семплувати випадкові рідкісні токени та уривки слів із навролонульовою базовою ймовірністю, повністю втрачаючи семантичний контроль.

Відповіді на питання (крок 8)

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Параметр temperature (T) масштабує логіти (сирі передбачення моделі) z_i перед їхнім перетворенням на ймовірності за допомогою функції softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Якщо $T = 1$, розподіл залишається незмінним. Якщо $T < 1$, знаменник дробу в показнику ступеня зменшується, що робить великі значення логітів ще більшими порівняно з меншими. Розподіл стає "гострішим": ймовірність найбільш вірогідних токенів стрімко наближається до 1, а інших — до 0. Якщо $T > 1$, розподіл "згладжується", наближаючись до рівномірного, що значно підвищує шанси вибору малої ймовірних або випадкових токенів.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Обидва параметри регулюють випадковість генерації, але працюють на різних етапах через різні механізми: temperature змінює саму форму розподілу ймовірностей для всього словника, математично перерозподіляючи "вагу" між лідерами та аутсайдерами перед генерацією. top_p (nucleus sampling) працює як фільтр або відсікання. Він сортує токени за спаданням ймовірності та залишає лише той мінімальний набір, сумарна ймовірність якого дорівнює або перевищує значення p. Усі інші токени відкидаються (отримують ймовірність 0), а залишені — перенормовуються до 100%. top_p обмежує пул можливих варіантів, але не змінює їхні відносні ймовірності між собою.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось

одне?

Відповідь: На практиці рекомендується налаштовувати лише один із цих параметрів, залишаючи інший за замовчуванням (як правило, `temperature = 1.0` або `top_p = 1.0`). Одночасна зміна обох ускладнює передбачення результату, оскільки ви одночасно і деформуєте форму розподілу, і обрізаєте його хвіст. Це може призвести або до надто жорстко обмеженого тексту, або до непередбачуваної хаотичності.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Хоча `temperature=0` змушує модель працювати в режимі жадібного декодування (завжди обирати токен із максимальною ймовірністю), це не гарантує повної відтворюваності. У файлі це підтверджується експериментально: при 10 запусках із `temperature=0` було зафіксовано 2 унікальні відповіді, тобто не всі вони були однаковими. Це зумовлено апаратними особливостями: Арифметика з плаваючою комою: Сучасні GPU і TPU виконують матричні множення через тисячі паралельних потоків. Оскільки операції з плаваючою комою не є строго асоціативними, зміна порядку додавання на мікрорівні при різних запусках може давати мікроскопічні похибки у фінальних логітах. Конкуренція токенів: Якщо два токени мають майже ідентичні логіти, ця апаратна мікропохибка може змінити токен-переможця, що потягне за собою зміну всього подальшого ланцюжка генерації.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних та фактажу: Використовують 0.0 – 0.2. Це мінімізує галюцинації та фіксує формат (JSON, таблиці). У файлі показано, що при `temperature=0.0` модель видала 5 із 5 правильних фактичних відповідей. Для творчого тексту (статті, ідеї, сценарії): Оптимальними є значення 0.7 – 0.9. Це забезпечує мовну гнучкість і цікаві повороти без втрати зв'язності. Дані з файлу свідчать, що при `temperature=0.7` генерується 5 унікальних творчих відповідей із 5. Однак екстремальні значення (наприклад, 1.5) призводять до повної втрати

точності — кількість правильних фактичних відповідей падає до 0, а текст стає незв'язним.

Висновок за завданням 6

Експеримент довів, що параметри генерації критично визначають баланс між фактологічною точністю та варіативністю відповіді. При мінімальних значеннях ($\text{temperature}=0.0$, $\text{top_p}=0.1$) модель забезпечує максимальну стабільність і безпомилковість у фактичних запитах, хоча навіть нульова температура не гарантує абсолютної відтворюваності через апаратний недетермінізм обчислень. Натомість агресивне зростання температури до 1.5 продемонструвало катастрофічну деградацію, перетворюючи вихідний текст на незв'язне псевдомовне марення внаслідок вирівнювання розподілу ймовірностей токенів. Хоча помірна стохастичність корисна для розширення вибірки ідей, фіксовані низькі пороги залишаються безальтернативними для фактажу та структурованих форматів на кшталт JSON, залишаючи високі значення виключно для вузької ніші некритичного творчого брейнштормінгу.

Загальні висновки

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	Від 14.4 до 245 символів за секунду.	Близько 96 токенів за секунду.	Від 71.8 до 655.4 символів за секунду.
TTFT	Від 2.05 до 2.08 секунд.	Повільніше, ніж мала модель.	Від 0.70 до 0.75 секунд.
Якість, балів /12	5/12	10/12	11/12

Стабільність формату (завдання 2)	Стабільно 2	Стабільно 2	Стабільно 2
Потрібні ресурси	Мінімальні, працює на звичайному процесорі.	Середні, потребує від 16 ГБ RAM для нормальної роботи.	Лише інтернет-з'єднання та API-ключ
Вартість	Безкоштовно	Безкоштовно	Є безкоштовні ліміти, далі оплата за використання.
Куди йдуть ваші дані	Залишаються на локальному комп'ютері.	Залишаються на локальному комп'ютері.	Передаються на сервери провайдера.
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для базового парсингу + приватність	Для завдань, що потребують логічного виведення та міркувань + приватність	Для складних завдань, що вимагають глибокого розуміння контексту та якісної української мови, якщо немає заборон на передачу даних.

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.